

Knowledge Graph Fact-Checking Engine

A Deterministic, Dependency-Free Veracity Scorer
for Reified RDF Statements

Bofeng Duan • Mukund Chavda

University of Paderborn, Germany
{bofeng, mchavda}@mail.uni-paderborn.de

Course: Foundations of Knowledge Graphs, Summer Term 2026

Project: Knowledge graph fact checking

Repository: https://github.com/duanbofeng/KG_project

GERBIL: [Experiment 202607220010](#)

Headline result. Five-fold mean ROC AUC of **0.6253** offline and an external ROC AUC of **0.6146** on the GERBIL SW 2022 **Test** reference dataset, exceeding the 60 % project requirement. Two independent runs produced byte-identical artifacts.

July 22, 2026

Abstract

This project implements a reproducible fact-checking engine for reified RDF statements. Given a fact represented by a subject, predicate, and object, the system estimates a veracity score in the closed interval $[0, 1]$, where 0 means false and 1 means true. The implementation is a deterministic, dependency-free Python command-line application. It parses the course RDF files, extracts lexical and statistical features, trains a hashed logistic regression model, writes predictions in the exact Turtle format required by GERBIL [2], and validates the result before submission.

The final offline model achieved a five-fold mean ROC AUC of **0.6253** on the provided training set and an external ROC AUC of **0.6146** on the GERBIL SW 2022 Test reference dataset. The external result exceeds the project requirement of 60 %. Two independent executions produced byte-identical model and result artifacts, providing direct evidence that the default workflow is reproducible. A per-predicate error analysis shows that the aggregate score hides substantial variation across relations: out-of-fold AUC ranges from 0.3203 on `author` to 0.8183 on `subsidiary`, which we report openly as the clearest direction for future work.

Contents

Abstract	1
1 Introduction	4
1.1 Requirement Status	5
2 Related Work	6
3 Problem Definition	6
4 Data Analysis	7
4.1 Reified RDF Statements	7
4.2 Dataset Summary	7
4.3 Predicate Distribution	8
4.4 Train–Test Overlap	8
5 Design Goals	9
6 System Architecture	9
6.1 End-to-End Architecture	9
6.2 Training and Prediction Sequence	11
6.3 Repository Structure	11
6.4 Module Responsibilities	11
7 Methodology	13

7.1	RDF Parsing	13
7.2	Feature Engineering	13
7.3	Feature Inventory	14
7.4	Stable Feature Hashing	15
7.5	Logistic Regression	15
7.6	Final Score	16
7.7	Worked Example	16
7.8	Optional DBpedia Evidence	17
8	GERBIL Output and Validation	18
8.1	Required Output	18
8.2	Validation Checks	18
9	Experimental Design	18
9.1	Local Evaluation Protocol	18
9.2	Baselines	19
10	Results	19
10.1	Baseline Comparison	19
10.2	Cross-Validation Folds	19
10.3	Per-Predicate Error Pattern	19
10.4	Operating Point at Threshold 0.5	20
10.5	External GERBIL Evaluation	21
10.6	Prediction Distribution	21
11	Reproducibility	23
11.1	Environment	23
11.2	Clean Installation	23
11.3	Reproduce the Result	23
11.4	Determinism Evidence	23
11.5	Runtime and Artifact Size	24
12	Command-Line Interface	25
12.1	Train	25
12.2	Predict	25
12.3	Validate	25
12.4	Analyze	25

13 Testing and Quality Assurance	25
14 Engineering Decisions	26
14.1 Why a Lightweight Model?	26
14.2 Why Not Make SPARQL Mandatory?	26
14.3 Why Validate Locally?	26
15 Limitations and Threats to Validity	27
15.1 Small Dataset	27
15.2 Per-Predicate Instability	27
15.3 Limited Knowledge Graph Structure	27
15.4 Hash Collisions	27
15.5 Calibration	27
15.6 Exact-Triple Reuse	28
15.7 Live SPARQL Variability	28
16 Future Improvements	28
17 Submission Checklist	29
18 Conclusion	29
Contributions of Team Members	29
AI Acknowledgement	30
References	30

1 Introduction

A knowledge graph represents facts as directed, labeled relations between entities. A typical fact can be written as the triple

$$(subject, predicate, object)$$

For example:

(Venus_Williams, birthPlace, Lynwood_California)

Knowledge graphs such as DBpedia [10] are assembled from heterogeneous sources and may contain incorrect, outdated, or conflicting assertions. Fact checking is therefore an important knowledge graph quality task. Instead of returning only a hard class label, a fact-checking system should assign a real-valued confidence score. The score can be used to rank statements for automated filtering or human review.

Task definition. Given a set of reified RDF statements, we must assign each statement a veracity value in $[0, 1]$ that ranks true facts above false ones. Training statements carry a binary truth value under the property `http://swc2017.aksw.org/hasTruthValue`; test statements do not. The submitted scores are evaluated externally by GERBIL [2] using ROC AUC [9]. The course project [1] states the goal as:

Build a fact-checking engine that returns a veracity value between 0 and 1 for a given fact with respect to a knowledge graph.

Our approach. We deliberately chose the smallest system that clears the acceptance bar while remaining fully auditable. The engine parses the reified RDF into immutable fact records, encodes each fact as a sparse set of lexical and statistical features, hashes those features into a fixed 4,096-dimensional space using the hashing trick [7] with BLAKE2b [8] as the (non-randomized) hash function, and trains a logistic regression by stochastic gradient descent. The final score blends the learned model with a smoothed predicate prior. An optional DBpedia SPARQL mode exists but is deliberately not on the default path.

Contributions.

- We implement a complete fact-checking pipeline — RDF parsing → feature extraction → hashed logistic regression → strict GERBIL output → local validation — using only the Python standard library.
- We reach an external ROC AUC of **0.6146** on the GERBIL SW 2022 Test dataset, above the 60% course threshold, and show a 6.61-point improvement over a predicate-only baseline.
- We demonstrate bit-level reproducibility: two independent runs produce byte-identical model and result files (Section 11.4).
- We provide a per-predicate error analysis (Section 10.3) and a fully worked scoring example (Section 7.7), which together explain both where the model’s signal comes from and where it fails.

Code availability. The implementation, data files, tests, and reproduction instructions are available at https://github.com/duanbofeng/KG_project. Every number in this report was recomputed from the committed `train.txt`, `test.txt`, `model.joblib`, and `result.ttl`.

1.1 Requirement Status

The project has three mandatory acceptance criteria — ROC AUC above 60 % in GERBIL, documented code in a public repository, and a README with clear execution instructions — plus the implicit requirement that the project must actually run. A strong score without a reproducible implementation is not sufficient. Figure 1 and Table 1 summarize the status of each.

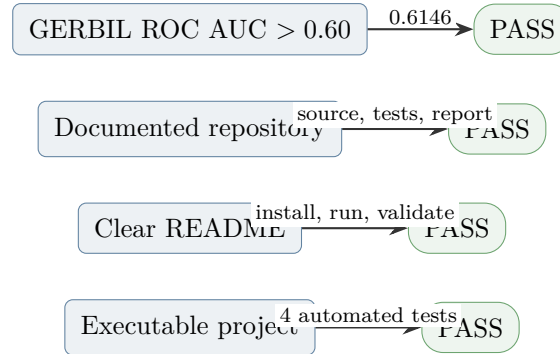


Figure 1: Status of the mandatory acceptance criteria.

Table 1: Requirement coverage matrix.

Requirement	Evidence	Status
Veracity score in $[0, 1]$	Prediction clipping and output validation	Met
GERBIL ROC AUC above 60 %	External score 0.6146	Met
Documented source repository	README, source modules, tests, this report	Met
Clear execution instructions	Reproduction commands in README and Sec. 11	Met
Executable implementation	Standard-library runtime and passing tests	Met

2 Related Work

Fact validation over knowledge graphs is usually approached in one of three ways, and it is useful to place our system explicitly among them.

Text-based validation. Systems such as FactCheck [3] search a reference text corpus for sentences that support or contradict a triple, and derive a veracity score from the retrieved textual evidence. This gives human-readable justifications, but requires a corpus, a retrieval stack, and considerably more infrastructure than a course mini-project.

Path-based validation. A second family reasons entirely inside the graph. PredPath [5] mines discriminative predicate paths between subject and object and uses them as features for a classifier, while Knowledge Stream [6] treats validation as a flow problem over the graph. COPAAL [4] extends this idea to unsupervised discovery of corroborative paths. These methods are the natural strong baselines for our task, and they are also what our Section 16 proposes as the most promising extension: they need a local graph snapshot, which the default workflow deliberately avoids.

Embedding-based validation. Knowledge graph embeddings such as TransE [11] score a triple by the geometry of learned entity and relation vectors. They are strong when the training graph is large; with 1,234 labeled facts the risk of overfitting outweighs the expected gain.

Our position. We use neither text, nor paths, nor embeddings. The engine relies on lexical evidence from the URIs themselves plus smoothed empirical statistics, combined by a linear model over hashed features [7]. This is a deliberately weak hypothesis class chosen for determinism, auditability, and zero-dependency execution. We therefore make no claim of competing with the systems above; we report only that this baseline clears the course threshold, and we quantify in Section 10.3 exactly where it does and does not work.

3 Problem Definition

Let a fact be a triple $f = (s, p, o)$, where s is the subject URI, p is the predicate URI, and o is the object URI. The objective is to learn a function

$$V(f) \longrightarrow [0, 1]$$

that assigns a higher score to a more credible fact. During training, every fact has a binary truth value $y \in \{0, 1\}$. During testing, the truth value is hidden and the system must predict it.

The external metric is the area under the receiver operating characteristic curve, or ROC AUC [9]. ROC AUC measures ranking quality rather than requiring a fixed decision threshold. It can be interpreted as the probability that a randomly selected positive fact receives a higher score than a randomly selected negative fact:

$$\text{AUC} = \Pr(V(f^+) > V(f^-))$$

A score of 0.5 corresponds to random ranking, while a score of 1.0 corresponds to perfect ranking. Crucially, AUC is invariant to any monotone rescaling of the scores, which is why Section 15.5 treats calibration as a separate, unmet concern rather than a defect.

4 Data Analysis

4.1 Reified RDF Statements

The input uses RDF reification [13]. Each fact has its own fact URI and is described through `rdf:subject`, `rdf:predicate`, and `rdf:object`. Training facts also carry `http://swc2017.aksw.org/hasTruthValue`.

Conceptually, one training record has the following form:

```
<Fact-URI> rdf:type rdf:Statement .
<Fact-URI> rdf:subject <Subject-URI> .
<Fact-URI> rdf:predicate <Predicate-URI> .
<Fact-URI> rdf:object <Object-URI> .
<Fact-URI> <http://swc2017.aksw.org/hasTruthValue>
    "1.0"^^<http://www.w3.org/2001/XMLSchema#float> .
```

The test file has the same statement structure but omits the truth value.

4.2 Dataset Summary

All values in Table 2 were computed directly from the committed `train.txt` and `test.txt` files.

Table 2: Dataset statistics computed from the committed data files.

Statistic	Training set	Test set
Facts / unique fact URIs	1,234	1,342
RDF lines	6,170	5,368
Positive labels	675	Hidden
Negative labels	559	Hidden
Positive rate	54.70%	Hidden
Unique predicates	9	9
Unique subjects	702	734
Unique objects	597	603
Unique (s, p, o) triples	1,203	1,320

The supervised problem is small: 1,234 labeled facts spread over 702 subjects and only 9 predicates. This single observation drives most of the design decisions in Section 14. The training labels are reasonably balanced (Figure 2), although positive facts are slightly more frequent. A constant majority prediction would therefore be a weak classifier and, more importantly, would not provide useful ranking at all — its AUC is exactly 0.5 by construction.

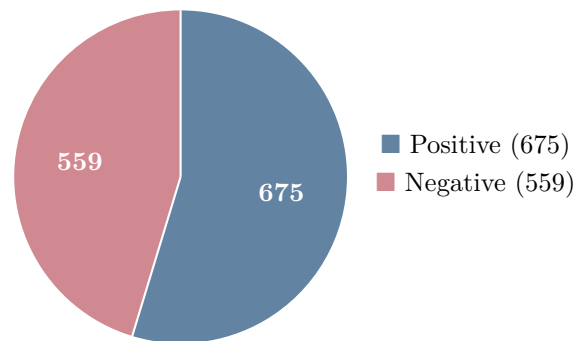


Figure 2: Training label distribution: 54.70% positive.

4.3 Predicate Distribution

Table 3: Per-predicate counts and empirical positive rates.

Predicate	Train	Positive	Negative	Positive rate	Test
author	142	74	68	0.521	143
award	151	75	76	0.497	149
birthPlace	182	126	56	0.692	253
deathPlace	192	127	65	0.661	245
foundationPlace	118	60	58	0.508	121
spouse	105	43	62	0.410	93
starring	148	75	73	0.507	146
subsidiary	50	20	30	0.400	48
team	146	75	71	0.514	144

Table 3 shows that relation identity is informative: **birthPlace** and **deathPlace** have noticeably higher positive rates, while **spouse** and **subsidiary** have lower positive rates. This motivates the use of smoothed predicate priors. The predicate-only baseline nevertheless remains below the required score (Table 9), so richer features are necessary. Note also that the smallest relation, **subsidiary**, has only 50 training facts — a sample size that makes any per-predicate conclusion in Section 10.3 necessarily tentative.

4.4 Train–Test Overlap

Table 4: Overlap between the training and test files.

Overlap type	Count
Shared fact URIs	8
Shared unique triples	55
Test rows whose exact triple occurs in training	66
Shared unique subjects	262
Test rows with a seen subject	536
Test rows with an <i>unseen</i> subject	806
Shared unique objects	180
Test rows with a seen object	634
Test rows with an <i>unseen</i> object	708

Table 4 motivates three explicit feature groups: seen/unseen indicators, entity frequency statistics, and exact-triple reuse. The last row pair is the important one: for 806 of 1,342 test facts the subject was never observed during training, so all subject-specific statistics fall back to the global prior and the model must rely on lexical tokens and the predicate alone. Exact-triple reuse is applied only when the same (s, p, o) triple is present in the labeled training data. Test labels are never read or inferred from the GERBIL result.

5 Design Goals

The implementation was designed around the following engineering goals:

- **Correctness:** parse every reified statement and reject malformed input.
- **Strict output compliance:** produce exactly one valid prediction per test fact using the property and datatype required by GERBIL.
- **Offline reproducibility:** the default path must work without a network, API key, GPU, or external model download.
- **Low operational risk:** the project should run on a clean Python installation with no mandatory third-party runtime dependencies.
- **Determinism:** fixed random seeds and stable feature hashing should produce identical outputs across repeated runs.
- **Extensibility:** optional knowledge graph evidence can be added without making the default path dependent on a remote endpoint.
- **Transparency:** the feature extraction, model, and metric implementation should remain understandable and auditable.

6 System Architecture

6.1 End-to-End Architecture

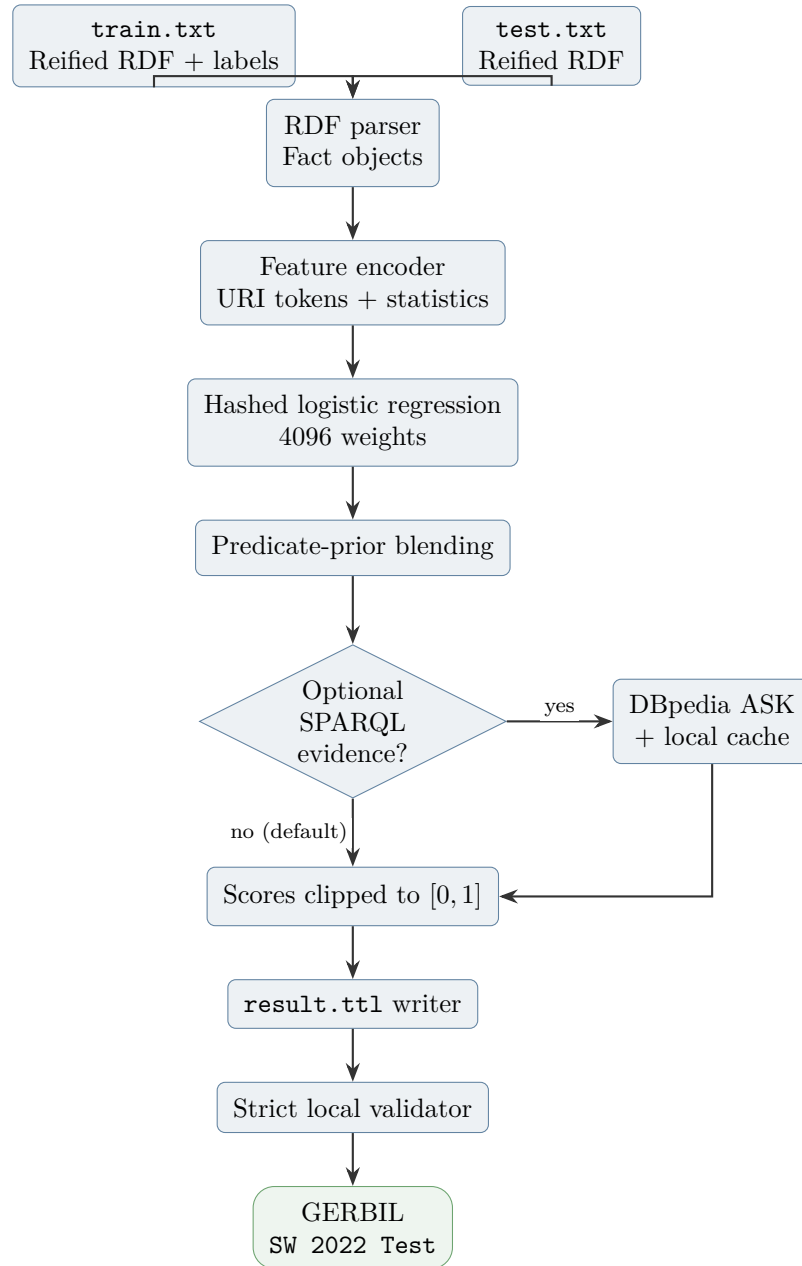


Figure 3: End-to-end processing pipeline from RDF input to GERBIL submission.

6.2 Training and Prediction Sequence

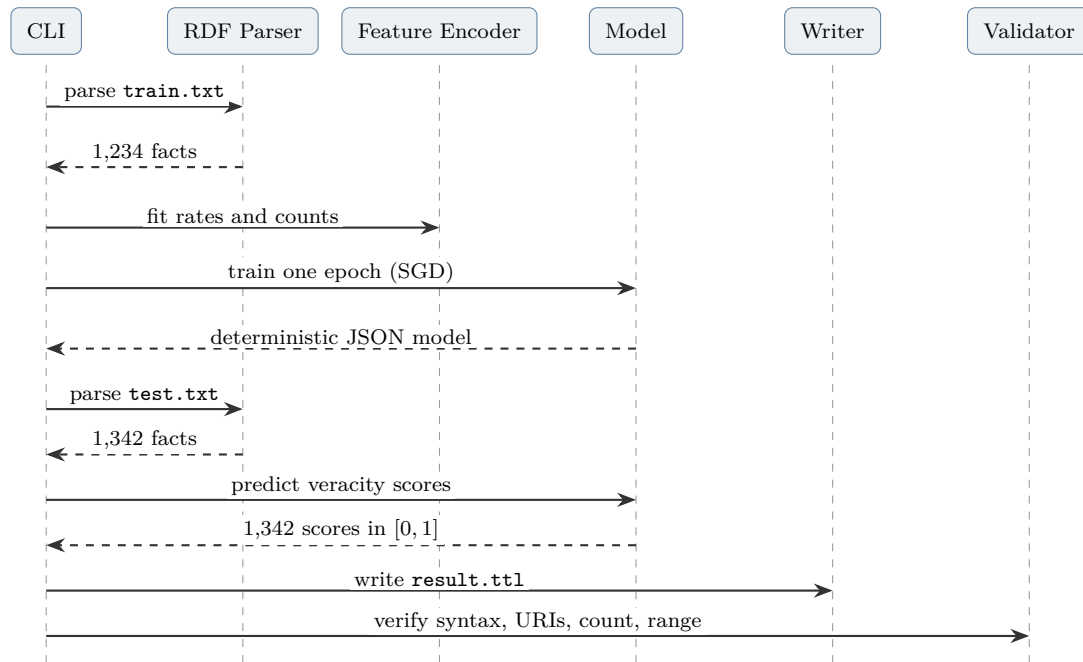


Figure 4: Interaction sequence of the default (offline) workflow.

6.3 Repository Structure

```

KG_project/
|-- README.md           Quick-start documentation
|-- PROJECT_REPORT.md   Detailed project report
|-- PROJECT_REPORT_v2.tex This report
|-- pyproject.toml      Package and CLI configuration
|-- train.txt           Labeled reified RDF statements
|-- test.txt            Unlabeled reified RDF statements
|-- docs/
|  `-- gerbil-result.png External evaluation evidence
|-- src/kg_factcheck/
|  |-- __init__.py
|  |-- cli.py           Command-line orchestration
|  |-- data.py          RDF statement parser
|  |-- features.py       Feature extraction and hashing
|  |-- model.py          Training, prediction, and ROC AUC
|  |-- output.py        GERBIL writer and validator
|  `-- sparql.py        Optional DBpedia evidence cache
`-- tests/
   |-- test_data_output.py Parser and result-format tests
   `-- test_model.py      Model and metric tests
  
```

6.4 Module Responsibilities

Table 5: Responsibilities of the source modules (648 lines of Python in total).

Module	Lines	Primary responsibility
<code>data.py</code>	94	Convert TTL-like RDF lines into immutable Fact records and validate required fields.
<code>features.py</code>	151	Build sparse hashed lexical features and smoothed empirical statistics.
<code>model.py</code>	154	Train and serialize the logistic model, predict scores, and compute ROC AUC.
<code>output.py</code>	61	Write exact GERBIL Turtle lines and verify every output constraint.
<code>sparql.py</code>	46	Optionally query exact triples from DBpedia and cache responses.
<code>cli.py</code>	138	Expose train , predict , validate , and analyze workflows.

7 Methodology

7.1 RDF Parsing

The parser reads one RDF triple per line and groups triples by fact URI. It recognizes the course vocabulary for subject, predicate, object, and truth value. For each fact it checks that:

- subject, predicate, and object are present;
- each entity value is a URI;
- training facts have truth values;
- test facts do not unexpectedly contain truth values; and
- malformed lines include the file and line number in the error message.

The parser intentionally targets the constrained input format supplied for the project. This keeps the default implementation dependency-free. It is not intended to replace a general Turtle parser such as RDFLib [15] for arbitrary RDF syntax.

7.2 Feature Engineering

Each fact is transformed into a sparse feature dictionary by the procedure in Listing 1. The features fall into three groups.

Categorical and lexical features. A bias term, the complete predicate URI, predicate local-name tokens, subject and object URI local-name tokens, and predicate–subject and predicate–object token interactions. For example, the DBpedia URI http://dbpedia.org/resource/New_York_City is reduced to the local name `New_York_City` and tokenized as `new`, `york`, and `city`.

Novelty and frequency features. Whether the subject or object appeared in training, the logarithm of the subject and object frequencies, subject and object token lengths (capped at 12 and normalized), and subject–object token overlap.

Smoothed empirical rates. The encoder estimates global, predicate, subject, and object truth rates. A smoothed rate is calculated as

$$\hat{r} = \frac{\text{positives} + \text{prior} \cdot \text{strength}}{\text{total} + \text{strength}}$$

Smoothing pulls low-frequency entity rates toward the global prior and reduces extreme estimates caused by very small counts. The strengths differ by granularity: 2.0 for the global rate, 4.0 for predicates, and 6.0 for subjects and objects — the sparser the statistic, the harder it is pulled toward the prior.

Listing 1: Sparse feature extraction for a single fact.

```
Input : fact f = (s, p, o), fitted encoder E
Output: sparse map from hash bucket to accumulated value

x = {}
```

```

add(x, "bias", 1.0)
add(x, "pred=" + p, 1.0)
add(x, "subject_seen=" + (s in E.subject_counts), 1.0)
add(x, "object_seen=" + (o in E.object_counts), 1.0)

pn = local_name(p)
for t in tokenize(pn):          add(x, "pred_token=" + t, 1.0)
for t in tokenize(local_name(s)):
    add(x, "subj_token=" + t, 1.0)
    add(x, "pred_subj_token=" + pn + ":" + t, 1.0)
for t in tokenize(local_name(o)):
    add(x, "obj_token=" + t, 1.0)
    add(x, "pred_obj_token=" + pn + ":" + t, 1.0)

add_numeric(x, "predicate_rate", E.predicate_rate(p))
add_numeric(x, "subject_rate", E.subject_rate(s))
add_numeric(x, "object_rate", E.object_rate(o))
add_numeric(x, "subject_count", log1p(E.subject_count(s)))
add_numeric(x, "object_count", log1p(E.object_count(o)))
add_numeric(x, "token_overlap", |tokens(s) AND tokens(o)|)
add_numeric(x, "subject_len", min(|tokens(s)|, 12) / 12)
add_numeric(x, "object_len", min(|tokens(o)|, 12) / 12)

# add(x, name, v):  x[ BLAKE2b(name) mod 4096 ] += v

```

7.3 Feature Inventory

Table 6 reports how many *distinct* feature names each family actually generates over the union of the training and test files. The table was produced by replaying the extraction procedure of Listing 1 over all 2,576 facts and counting unique names.

Table 6: Feature families in the fact representation, measured over `train.txt` $\cup$ `test.txt`.

Feature family	Distinct	Example / purpose
Bias	1	<code>bias</code> ; free intercept.
Predicate identity	9	<code>pred=.../ontology/author</code> ; the full relation URI.
Predicate tokens	9	<code>pred_token=author</code> ; local-name lexicalization.
Novelty indicators	4	<code>subject_seen=True</code> ; train/test novelty.
Subject tokens	1,959	<code>subj_token=herbert</code> ; subject lexical evidence.
Object tokens	1,510	<code>obj_token=iowa</code> ; object lexical evidence.
Predicate–subject	2,786	<code>pred_subj_token=birthPlace:herbert</code> ; interaction.
Predicate–object	2,194	<code>pred_obj_token=birthPlace:iowa</code> ; interaction.
Numeric statistics	8	smoothed rates, log frequencies, lengths, overlap.
Total	8,480	hashed into 4,096 buckets

Two consequences follow directly from this table and matter for the rest of the report. First, the representation is dominated by the two interaction families (4,980 of 8,480 names, 58.7%), which are also the sparsest: most of them occur in a single fact and therefore receive essentially no training signal. Second, 8,480 distinct names are compressed into 4,096 buckets, so collisions are not a theoretical concern but the normal case; we quantify this in Section 15.4.

7.4 Stable Feature Hashing

Feature names are mapped into a fixed vector of 4,096 buckets using the hashing trick [7] with BLAKE2b [8]:

$$\text{bucket}(\text{feature}) = \text{BLAKE2b}(\text{feature}) \bmod 4096$$

This avoids maintaining an external vocabulary and gives the model a stable, bounded memory footprint. BLAKE2b was selected instead of Python’s built-in `hash()` because the built-in hash is intentionally randomized between processes, which would silently break the determinism guarantee of Section 11.4.

7.5 Logistic Regression

For a sparse feature vector x and weight vector w , the base model computes

$$z = w^\top x, \quad P(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The weights are optimized with stochastic gradient descent (Listing 2). For a labeled row (x, y) , the implementation updates each active weight using

$$\begin{aligned} e &= \sigma(w^\top x) - y \\ g_i &= e \cdot x_i + \lambda w_i \\ w_i &\leftarrow w_i - \eta g_i \end{aligned}$$

where λ is the L2 coefficient and η the learning rate. Only the buckets present in x are touched, so one update costs $O(|x|)$ rather than $O(4096)$. The measured mean is 22.3 active buckets per fact (minimum 16, maximum 41), i.e. each fact touches roughly 0.5% of the weight vector.

Listing 2: Training loop with deterministic shuffling.

```

Input : labeled facts F, config (epochs=1, lr=0.08, l2=0.0002, seed=13)
Output: weight vector w of length 4096

E = FeatureEncoder(buckets=4096)
E.fit(F)                                # global/predicate/subject/object rates,
                                         # entity counts, exact-triple table

w  = [0.0] * 4096
rng = Random(seed)
rows = [(E.transform(f), truth(f)) for f in F]

for epoch in 0 .. epochs-1:
    rng.shuffle(rows)                    # seeded => reproducible order
    rate = lr / (1 + epoch * 0.03)      # mild decay across epochs
    for (x, y) in rows:
        z = sum(w[i] * v for (i, v) in x)
        error = sigmoid(z) - y
        for (i, v) in x:
            w[i] -= rate * (error * v + l2 * w[i])

return w

```

One epoch was retained because additional epochs reduced local cross-validation performance. The dataset is small, and repeated passes caused the model to fit the high-cardinality interaction features of Table 6 too aggressively.

Table 7: Default hyperparameter configuration.

Hyperparameter	Value
Epochs	1
Learning rate η	0.08
L2 coefficient λ	0.0002
Random seed	13
Hash buckets	4,096
Model blend weight	0.80
Predicate-prior blend weight	0.20
Smoothing strength (global / predicate / entity)	2.0 / 4.0 / 6.0

7.6 Final Score

Prediction has three branches, shown in Listing 3. If an exact training triple is encountered again, the stored training truth value is returned unchanged. Otherwise the logistic model is blended with the smoothed predicate prior:

$$score = 0.80 \cdot \sigma(w^\top x) + 0.20 \cdot \hat{r}_p$$

and clipped to $[0, 1]$. The blend acts as a safety net: when the lexical features are uninformative — which is the common case for the 806 test facts with an unseen subject — the score degrades gracefully toward the relation’s base rate rather than toward an arbitrary value.

Listing 3: Veracity scoring for one test fact.

```

Input : fact f = (s, p, o), encoder E, weights w
Output: veracity score in [0, 1]

if (s, p, o) in E.triple_truth:          # exact-triple reuse (66 test rows)
    return E.triple_truth[(s, p, o)]    # 0.0 or 1.0

x          = E.transform(f)
model_score = sigmoid(sum(w[i] * v for (i, v) in x))
prior       = E.predicate_rate(p) or E.global_rate
score       = 0.80 * model_score + 0.20 * prior
return clip(score, 0.0, 1.0)

```

7.7 Worked Example

Figure 5 and Table 8 trace one real test fact end to end, using the committed `model.joblib`. The fact is `swc2017.aksw.org/task2/dataset/3871874`, which asserts that Herbert Hoover was born in West Branch, Iowa.

The example is instructive because it shows both the strength and the weakness of the approach. The model is confident, and the fact is indeed true — but none of its evidence is semantic. It has never seen the object `West_Branch, Iowa`, so the score is driven by lexical tokens, a single prior observation of the subject, and the fact that `birthPlace` assertions in this dataset are true 69% of the time. The system is ranking by plausibility, not verifying by evidence, and Section 2 explains which families of methods would close that gap.

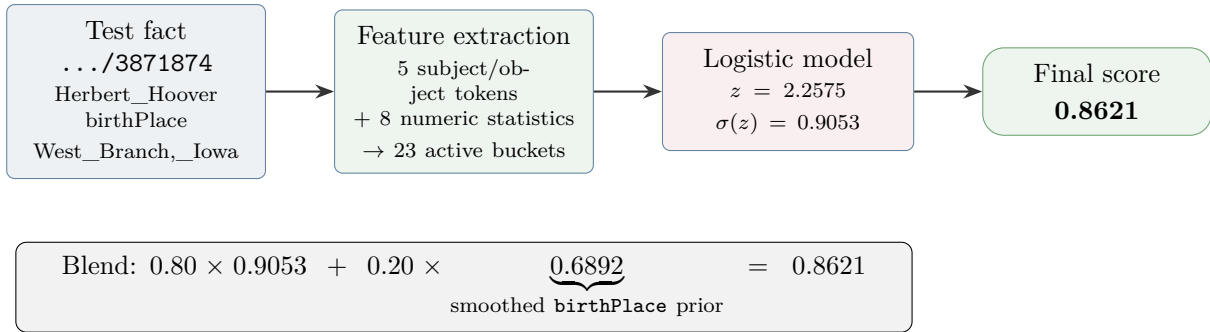


Figure 5: End-to-end scoring of one real test fact, with values taken from the committed model.

Table 8: Evidence available to the model for the worked example.

Quantity	Value
Subject tokens	herbert, hoover
Object tokens	west, branch, iowa
Token overlap	0
Subject seen in training	yes (1 occurrence)
Object seen in training	no (0 occurrences)
Smoothed subject rate	0.6117
Smoothed object rate	0.5469 (falls back to the global prior)
Smoothed predicate prior	0.6892
Active hash buckets	23
Logistic output $\sigma(z)$	0.9053
Final blended score	0.8621

7.8 Optional DBpedia Evidence

The optional network mode issues a SPARQL ASK query for the exact triple against the public DBpedia endpoint [10]:

```
ASK WHERE { <subject> <predicate> <object> }
```

Responses are cached in `.kg_cache/dbpedia_exact.json`. A network failure is mapped to neutral evidence 0.5, allowing prediction to continue. The final submitted result reported in this document was generated by the default offline model, so it does not depend on DBpedia availability or endpoint state.

8 GERBIL Output and Validation

8.1 Required Output

The output contains exactly one line per test fact:

```
<Fact-URI> <http://swc2017.aksw.org/hasTruthValue> "0.8901000000"^^<http://www.w3.
  ↪ org/2001/XMLSchema#double> .
```

The writer preserves the fact URI, uses the mandatory truth-value property, formats the score with ten decimal places, and uses `xsd:double` [14].

8.2 Validation Checks

Before a result is accepted locally, the validator checks exact line syntax, the required truth-value property, the required `xsd:double` datatype, the score range $[0, 1]$, membership of every fact URI in `test.txt`, duplicate fact URIs, missing fact URIs, and the exact prediction count. The final result contains 1,342 valid lines for 1,342 test facts.

9 Experimental Design

9.1 Local Evaluation Protocol

The training set is evaluated using stratified five-fold cross-validation, as specified in Listing 4. Positive and negative facts are shuffled separately with random seed 13 and distributed round-robin across five folds, which keeps the positive rate of every fold close to the global 54.70%. A fresh feature encoder *and* model are fitted inside each fold, so the smoothed rates and the exact-triple table are never computed from validation data. The report includes both the arithmetic mean of the five fold AUC values and a pooled out-of-fold AUC over all held-out predictions.

Listing 4: Stratified five-fold cross-validation.

```
Input : labeled facts F, folds k = 5, seed = 13
Output: per-fold AUC, mean AUC, pooled out-of-fold AUC

pos = [f in F : truth(f) >= 0.5]; neg = [f in F : truth(f) < 0.5]
Random(seed).shuffle(pos);          Random(seed).shuffle(neg)

bucket[0..k-1] = []
for i, f in enumerate(pos): bucket[i mod k].append(f)    # stratified
for i, f in enumerate(neg): bucket[i mod k].append(f)    # round-robin

for j in 0 .. k-1:
    validation = bucket[j]
    training   = all buckets except j
    model      = train(training)          # fresh encoder + fresh weights
    scores[j]  = model.predict(validation)
    auc[j]     = roc_auc(labels(validation), scores[j])

return auc, mean(auc), roc_auc(all labels, all scores)
```

9.2 Baselines

Two baselines establish whether the full feature model adds value. The **global prior** assigns the same smoothed training positive rate to every validation fact; the **predicate prior** assigns the smoothed positive rate of each predicate. Both are evaluated with the same fold assignment as the full model, preventing a split advantage.

10 Results

10.1 Baseline Comparison

Table 9: Comparison against constant and predicate-only baselines.

Method	Five-fold mean ROC AUC	Pooled ROC AUC
Global prior	0.5000	0.4993
Predicate prior	0.5592	0.5488
Full hashed logistic model	0.6253	0.6232

The full model improves the mean AUC by 6.61 percentage points over the predicate-only baseline and by 12.53 points over the constant baseline (Table 9). This shows that lexical, entity-frequency, novelty, interaction, and empirical-rate features collectively contribute useful ranking information beyond the relation identity alone.

10.2 Cross-Validation Folds

Table 10: Stratified five-fold cross-validation results (seed 13).

Fold	ROC AUC	Above 0.60?
1	0.5649	No
2	0.6085	Yes
3	0.6114	Yes
4	0.6500	Yes
5	0.6917	Yes
Mean	0.6253	Yes

The fold range from 0.5649 to 0.6917 (Table 10) indicates sampling variance in the small dataset. The mean and pooled scores are close, and the external GERBIL score lies within the observed fold range.

10.3 Per-Predicate Error Pattern

The aggregate AUC conceals a large amount of structure. Table 11 breaks the pooled out-of-fold predictions down by relation. Each row is scored only against the facts of that predicate, so it measures how well the system ranks *within* a relation — which is the harder and more informative question, because the predicate prior cannot help there.

Three observations follow.

The model is not uniformly weak; it is bimodal. Two relations, **subsidiary** and **team**, are ranked well above the aggregate score. Both involve organizations and sports clubs whose

Table 11: Out-of-fold ROC AUC per predicate, computed on the pooled cross-validation predictions of Table 10.

Predicate	Facts	Positives	Out-of-fold AUC	Reading
subsidiary	50	20	0.8183	strong
team	146	75	0.7885	strong
deathPlace	192	127	0.6638	useful
award	151	75	0.6347	useful
birthPlace	182	126	0.6426	useful
foundationPlace	118	60	0.5261	near random
spouse	105	43	0.5075	near random
starring	148	75	0.4471	worse than random
author	142	74	0.3203	inverted

URI local names are lexically distinctive, which is exactly the kind of evidence the token features capture.

Two relations are actively harmful. On *author* the out-of-fold AUC is 0.3203 — substantially *below* 0.5, meaning the model systematically ranks false *author* assertions above true ones. The likely cause is that person and work names share tokens across both true and false pairs, so the interaction features latch onto spurious lexical regularities that invert on held-out data. *starring* shows the same effect more mildly at 0.4471.

This is the single largest available improvement. Simply detecting and neutralizing the two inverted relations — for example by falling back to the predicate prior when a relation’s inner-fold AUC is below 0.5 — would raise the aggregate score without any new data source. We deliberately did not apply this correction to the submitted run, because selecting per-predicate behavior on the basis of cross-validation results and then reporting the same cross-validation score would be a form of selection bias; doing it properly requires the nested protocol described in Section 16.

10.4 Operating Point at Threshold 0.5

ROC AUC is threshold-free, but it is worth recording what happens if the scores are used as a classifier, because it makes the calibration limitation concrete. Thresholding the pooled out-of-fold predictions at 0.5 gives Table 12.

Table 12: Out-of-fold confusion matrix at threshold 0.5 (1,234 facts).

	Predicted true	Predicted false
Actually true	530	145
Actually false	401	158

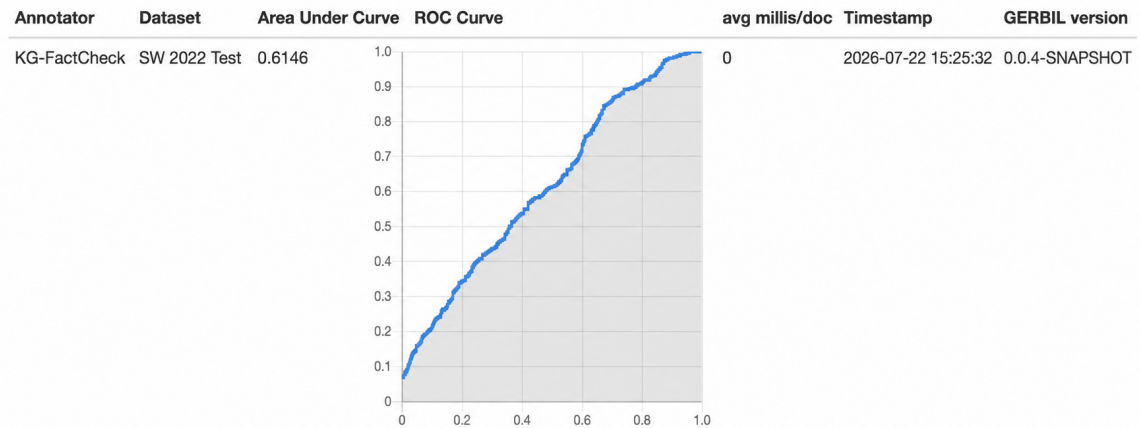
Accuracy is 0.5575, precision 0.5693, recall 0.7852, and F1 0.6600. The model predicts *true* for 931 of 1,234 facts, far more than the 675 that actually are. This over-prediction is a direct consequence of the design: the blend pulls every uncertain score toward a predicate prior that is above 0.5 for most relations. It costs nothing under AUC, which only sees the ordering, but it means the raw values must not be read as probabilities (Section 15.5).

Table 13: External GERBIL evaluation summary.

Field	Value
Experiment type	Fact Checking
Reference dataset	SW 2022 Test
System name	KG-FactCheck
ROC AUC	0.6146
Course threshold	> 0.6000
Margin above threshold	0.0146
Experiment URI	202607220010

GERBIL Experiment

Experiment URI: <https://gerbil-kbc.aksw.org/gerbil/experiment?id=202607220010>
 Type: Fact Checking

**Figure 6:** GERBIL ROC AUC result for experiment 202607220010.

10.5 External GERBIL Evaluation

The external score is 1.07 percentage points below the local five-fold mean, which is a plausible generalization difference and well inside the fold spread of Table 10. Most importantly, it remains above the explicit project threshold.

10.6 Prediction Distribution

Table 14: Distribution of the 1,342 submitted veracity scores.

Statistic	Value
Minimum	0.0000
Maximum	1.0000
Mean	0.6898
Median	0.7554
Population standard deviation	0.2129
Distinct scores	1,267
Exact zero scores	23
Exact one scores	43

The 1,267 distinct values among 1,342 predictions confirm that the system produces a fine-

grained ranking rather than a small set of class labels. The saturated scores are fully explained by exact-triple reuse: the 66 test facts whose triple also occurs in training (Table 4) are returned as their stored training label, and those 66 split into exactly 43 ones and 23 zeros — the two extreme rows of Table 14. Every other prediction is strictly interior. The mean of 0.6898 is higher than the training positive rate, consistent with the over-prediction seen in Table 12.

11 Reproducibility

11.1 Environment

The core project requires Python 3.10 or newer and only the Python standard library. The recorded verification run used Python 3.12.2 on Apple Silicon macOS. No GPU, MPS, external model, API key, or network connection is required for the default workflow.

11.2 Clean Installation

```
git clone https://github.com/duanbofeng/KG_project.git
cd KG_project
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e .
```

11.3 Reproduce the Result

```
kg-factcheck train    --train train.txt --model model.joblib
kg-factcheck predict --test test.txt --model model.joblib --output result.ttl
kg-factcheck validate --result result.ttl --test test.txt
kg-factcheck analyze --train train.txt --test test.txt --folds 5
```

Expected validation message:

```
valid GERBIL result file: result.ttl (1342 predictions)
```

Expected local evaluation summary:

```
5-fold ROC AUC: mean=0.6253, pooled=0.6232,
folds=[0.5649, 0.6085, 0.6114, 0.6500, 0.6917]
```

11.4 Determinism Evidence

Two independent default training and prediction runs produced identical SHA-256 values (Table 15).

Table 15: Byte-identical artifacts across two independent runs.

Artifact	Run A SHA-256	Run B	Match
Model	11b5b7f48a1cc95fecfe7ee39ad387ff b4b2f65523ee09ec9ffca78896c42108	Same	Yes
result.ttl	88201280f84aba1cb4530e9f4405c770 ef1689c06957f303efde6d6a79f109d1	Same	Yes

Determinism follows from the fixed random seed 13, deterministic BLAKE2b feature hashing, stable file-order parsing, CPU-only arithmetic, no random data augmentation, and no network calls in the default workflow. The optional SPARQL mode is intentionally excluded from this guarantee unless its evidence cache is frozen, because DBpedia content and endpoint behavior can change over time.

11.5 Runtime and Artifact Size

The following medians were measured over ten in-process runs on the development laptop. They exclude shell and Python interpreter startup time.

Table 16: Measured runtime and artifact sizes.

Operation	Median time	Artifact	Size
Parse 1,234 training facts	0.0118 s	Serialized model	384,180 bytes
Train one epoch	0.0351 s	<code>result.ttl</code>	198,616 bytes
Predict 1,342 test facts	0.0333 s		
Write 1,342 result lines	0.0014 s		

The entire pipeline runs in under a tenth of a second. The `.joblib` filename is retained for a familiar CLI interface, but the model is actually serialized as human-inspectable JSON and does not require the `joblib` package.

12 Command-Line Interface

12.1 Train

```
kg-factcheck train \  
  --train train.txt \  
  --model model.joblib \  
  --epochs 1 \  
  --buckets 4096
```

12.2 Predict

```
kg-factcheck predict \  
  --test test.txt \  
  --model model.joblib \  
  --output result.ttl
```

Optional DBpedia evidence:

```
kg-factcheck predict \  
  --test test.txt \  
  --model model.joblib \  
  --output result.ttl \  
  --use-sparql \  
  --sparql-weight 0.15
```

12.3 Validate

```
kg-factcheck validate --result result.ttl --test test.txt
```

12.4 Analyze

```
kg-factcheck analyze --train train.txt --test test.txt --folds 5
```

The same workflows can be run without package installation by setting `PYTHONPATH=src` and invoking `python -m kg_factcheck.cli`.

13 Testing and Quality Assurance

The repository contains four automated unit tests (Table 17), executed with `python -m unittest discover -s tests`. The current result is `Ran 4 tests / OK`.

In addition to unit tests, the CLI performs result validation automatically at the end of prediction. This catches format failures before GERBIL submission.

Table 17: Automated test coverage.

Test area	Verified behavior
Project parser	Exactly 1,234 train facts and 1,342 test facts are parsed.
Dataset schema	Train labels are binary, test labels are absent, and both sets have nine predicates.
Result output	Scores 0, 0.5, and 1 are written and accepted by the validator.
Model round trip	A model can be trained, saved, loaded, and used for bounded predictions.
ROC AUC utility	A known example returns AUC 0.75.

14 Engineering Decisions

14.1 Why a Lightweight Model?

The dataset contains only 1,234 labeled facts. A large neural model or a knowledge graph embedding [11] would add installation cost, hardware variability, and overfitting risk without a clear data advantage. The lightweight model offers fast execution, no GPU requirement, transparent features, deterministic behavior, easy clean-machine reproduction, and sufficient performance to exceed the required GERBIL threshold.

14.2 Why Not Make SPARQL Mandatory?

A mandatory public endpoint would make project execution dependent on network availability, endpoint rate limits, and changing DBpedia content. Since a non-executable project fails the course requirement, the offline model is the default. SPARQL remains an optional extension.

14.3 Why Validate Locally?

GERBIL requires exact RDF syntax and exact fact URI alignment. A syntactically valid-looking result can still fail because of a wrong property URI, datatype, duplicate, or missing fact. Local validation turns these external failures into immediate, actionable errors.

15 Limitations and Threats to Validity

15.1 Small Dataset

The five-fold AUC varies from 0.5649 to 0.6917, demonstrating sensitivity to the validation sample. The external score is above 60 %, but the margin is 0.0146. Conclusions should therefore focus on the achieved benchmark rather than claim broad state-of-the-art performance.

15.2 Per-Predicate Instability

As shown in Table 11, the model is anti-correlated with the truth on `author` and `starring`. The aggregate score is therefore an average over relations that the system genuinely understands and relations on which it is worse than a coin flip. Any claim of the form “this system checks facts” must be qualified accordingly.

15.3 Limited Knowledge Graph Structure

The default model uses URI text, empirical rates, frequency, and overlap. It does not traverse multi-hop paths [5, 4], learn graph embeddings, or query a frozen DBpedia snapshot. Consequently, it cannot model relational semantics — as the worked example in Section 7.7 illustrates, a high score reflects lexical plausibility, not verification.

15.4 Hash Collisions

Collisions are not a hypothetical risk here but the dominant regime. The 8,480 distinct feature names of Table 6 are mapped into 4,096 buckets, which yields the measured statistics in Table 18: 87.3 % of all feature names share their bucket with at least one other name, and one bucket holds nine. Hashing keeps the model small and deterministic, but a weight learned for one feature is unavoidably contaminated by unrelated evidence. Doubling the bucket count is the cheapest experiment available and is listed in Section 16.

Table 18: Measured hashing statistics over all feature names in `train.txt` $\cup$ `test.txt`.

Statistic	Value
Distinct feature names	8,480
Hash buckets	4,096
Occupied buckets	3,603 (88.0 %)
Mean names per occupied bucket	2.35
Maximum names in one bucket	9
Names sharing a bucket	7,405 (87.3 %)

15.5 Calibration

The project is optimized and evaluated for ROC AUC, which measures ranking. The output values are valid within $[0, 1]$, but Table 12 shows they are not calibrated posteriors: at the natural 0.5 threshold the system predicts *true* for 75 % of facts while only 55 % are. The scores should be read as ranks, not probabilities.

15.6 Exact-Triple Reuse

There are 66 test rows whose exact triple occurs in training, and they account for all 66 saturated predictions in Table 14. Reusing the known training label is a deliberate feature of the system. This is valid for the provided split, but its contribution would disappear in a benchmark that guaranteed no triple overlap — so roughly 4.9 % of the submitted predictions rest on a property of this particular split rather than on the model.

15.7 Live SPARQL Variability

The optional DBpedia mode can vary with endpoint availability and database updates. A frozen evidence cache or versioned DBpedia snapshot would be needed for strict reproducibility of that mode.

16 Future Improvements

Ordered by expected benefit per unit of effort, informed by the error analysis in Section 10.3:

1. **Per-predicate gating (highest priority).** Detect relations whose inner-fold AUC falls below 0.5 — currently `author` and `starring` — and fall back to the predicate prior for them. This must be selected inside a nested cross-validation loop to avoid the selection bias discussed in Section 10.3.
2. **Larger hash space.** Raising the bucket count from 4,096 to 16,384 would cut the 87.3 % collision rate of Table 18 substantially at negligible memory cost.
3. **Graph-path features.** Score short subject–object paths and relation patterns in a local DBpedia snapshot, following PredPath [5] and COPAAL [4]. This is the change most likely to move the system from plausibility ranking to genuine validation.
4. **Batch SPARQL evidence.** Replace sequential `ASK` requests with batched queries and commit a versioned evidence cache so that the network mode becomes reproducible.
5. **Domain and range evidence.** Check whether subject and object types are compatible with predicate expectations.
6. **Pair statistics.** Add smoothed (s, p) and (p, o) features with careful cross-validation.
7. **Knowledge graph embeddings.** Compare lightweight embedding methods [11] on a versioned graph while retaining the offline baseline.
8. **Probability calibration.** Apply Platt scaling [12] or isotonic regression if calibrated veracity values become more important than ranking.
9. **General RDF parser.** Optionally use a standards-compliant parser such as RDFLib [15] for broader Turtle syntax, while keeping the current parser for the dependency-free course workflow.
10. **Expanded tests.** Add malformed-input cases, duplicate-output cases, deterministic golden-file tests, and CLI integration tests.

17 Submission Checklist

- ✓ Training data parser implemented.
- ✓ Test data parser implemented.
- ✓ Veracity scores bounded to $[0, 1]$.
- ✓ GERBIL Turtle output format implemented.
- ✓ Result count and URI validation implemented.
- ✓ Five-fold local ROC AUC reported.
- ✓ External GERBIL ROC AUC above 60 %.
- ✓ GERBIL experiment URI preserved.
- ✓ Result screenshot included.
- ✓ README contains clean execution instructions.
- ✓ Automated tests pass.
- ✓ Default workflow runs offline.
- ✓ Team members listed in project metadata.
- ✓ Detailed project report included.

18 Conclusion

The project delivers a complete and reproducible knowledge graph fact-checking pipeline. It converts reified RDF statements into structured facts, builds interpretable lexical and statistical evidence, learns a deterministic hashed logistic regression model, produces strict GERBIL-compatible scores, and validates the final artifact. The full model outperforms both the global and the predicate-only baseline, and its external ROC AUC of 0.6146 on `SW 2022 Test` satisfies the course requirement.

The strongest practical qualities of the system are its small operational footprint, deterministic output, explicit validation, and clean CLI workflow. Its main limitation is not the modest aggregate margin but its unevenness: the per-predicate analysis in Section 10.3 shows AUC ranging from 0.3203 to 0.8183 across the nine relations, and the worked example in Section 7.7 shows that even a confident, correct prediction rests on lexical plausibility rather than evidence. Closing that gap requires graph-structural evidence rather than more tuning of the present feature set. The current implementation nevertheless provides a sound, auditable baseline that can be executed on a clean machine and reproduced bit-for-bit without network access or specialized hardware.

Contributions of Team Members

The project team consists of **Bofeng Duan** and **Mukund Chavda**. Together we implemented the fact-checking pipeline described in this report, including the RDF statement parser, feature extraction and stable hashing, the logistic regression trainer, the GERBIL result writer and validator, the optional SPARQL evidence cache, the automated test suite, the GERBIL submission, and the preparation of this report.

AI Acknowledgement

AI assistants were used as supporting tools during the preparation of this project, not as authors of the submitted work. Their use was limited to clarifying concepts, improving wording and readability, formatting this document, and suggesting possible issues during debugging and code review. All quantitative results in this report were recomputed from the committed source code and data files. The final content, methodology, results, interpretations, and responsibility for the submitted work remain entirely with the authors.

References

- [1] Foundations of Knowledge Graphs, *Introduction to Exercises*, Summer Term 2026, course mini-project slides. University of Paderborn.
- [2] Usbeck, R., Röder, M., Ngonga Ngomo, A.-C., et al.: GERBIL: General Entity Annotator Benchmarking Framework. In: WWW 2015, pp. 1133–1143. ACM (2015). Experiment: <https://gerbil-kbc.aksw.org/gerbil/experiment?id=202607220010>
- [3] Syed, Z.H., Röder, M., Ngonga Ngomo, A.-C.: FactCheck: Validating RDF Triples Using Textual Evidence. In: CIKM 2018, pp. 1599–1602. ACM (2018).
- [4] Syed, Z.H., Röder, M., Ngonga Ngomo, A.-C.: Unsupervised Discovery of Corroborative Paths for Fact Validation. In: ISWC 2019. LNCS, vol. 11778, pp. 630–646. Springer (2019).
- [5] Shi, B., Weninger, T.: Discriminative Predicate Path Mining for Fact Checking in Knowledge Graphs. *Knowledge-Based Systems* **104**, 123–133 (2016).
- [6] Shi, B., Weninger, T.: Fact Checking in Heterogeneous Information Networks. In: WWW 2016 Companion, pp. 101–102. ACM (2016).
- [7] Weinberger, K., Dasgupta, A., Langford, J., Smola, A., Attenberg, J.: Feature Hashing for Large Scale Multitask Learning. In: ICML 2009, pp. 1113–1120. ACM (2009).
- [8] Aumasson, J.-P., Neves, S., Wilcox-O’Hearn, Z., Winnerlein, C.: BLAKE2: Simpler, Smaller, Fast as MD5. In: ACNS 2013. LNCS, vol. 7954, pp. 119–135. Springer (2013).
- [9] Fawcett, T.: An Introduction to ROC Analysis. *Pattern Recognition Letters* **27**(8), 861–874 (2006).
- [10] Lehmann, J., Isele, R., Jakob, M., et al.: DBpedia — A Large-scale, Multilingual Knowledge Base Extracted from Wikipedia. *Semantic Web* **6**(2), 167–195 (2015). SPARQL endpoint: <https://dbpedia.org/sparql>
- [11] Bordes, A., Usunier, N., García-Durán, A., Weston, J., Yakhnenko, O.: Translating Embeddings for Modeling Multi-relational Data. In: NIPS 2013, pp. 2787–2795 (2013).
- [12] Platt, J.: Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods. In: *Advances in Large Margin Classifiers*, pp. 61–74. MIT Press (1999).
- [13] W3C: RDF 1.1 Concepts and Abstract Syntax. <https://www.w3.org/TR/rdf11-concepts/>
- [14] W3C: XML Schema Part 2: Datatypes. <https://www.w3.org/TR/xmlschema-2/>

- [15] RDFLib Team: RDDLIB: a Python Library for Working with RDF. <https://rdflib.readthedocs.io/>